2

	8	

1

1

```
0780:0020 8BFE          MOV     DI,SI
0780:0022 83C602        ADD     SI,+02
0780:0025 E2E8          LOOP    000F
0780:0027 891E2000      MOV     [0020],BX
0780:002B 893E2200      MOV     [0022],DI
```

```
-u
0780:0000 B87E07        MOV     AX,077E
0780:0003 8ED8          MOV     DS,AX
0780:0005 B91000        MOV     CX,0010
0780:0008 33F6          XOR     SI,SI
0780:000A 33FF          XOR     DI,DI
0780:000C BB0000        MOV     BX,0000
0780:000F 8B840000      MOV     AX,[SI+0000]
0780:0013 3D0000        CMP     AX,0000
0780:0016 7F02          JG      001A
0780:0018 F7D8          NEG     AX
0780:001A 3BC3          CMP     AX,BX
0780:001C 7C04          JL      0022
0780:001E 8BD8          MOV     BX,AX
```

2 M n+2 n+2

```
-d 0 23
077E:0000  22 00 36 00 11 00 49 00-68 00 06 00 07 00 08 00
077E:0010  09 00 10 00 11 00 12 00-13 00 14 00 15 00 16 00
077E:0020  B8 7E 07 8E
```

3 M n+2 n+2

4

```
-d 0 23
077E:0000  22 00 36 00 11 00 49 00-68 00 06 00 07 00 08 00
077E:0010  09 00 10 00 11 00 12 00-13 00 14 00 15 00 16 00
077E:0020  68 00 08 00
```

```
        ; empty asm file

title I love asm

data segment
        M    dw      22h, 36h, 11h, 49h, 68h, 6h, 7h, 8h, 9h, 10h, 11h, 12h, 13h,
14h, 15h, 16h
      n    equ    ($ - M) / 2
data ends

code segment
        assume cs:code, ds:data
        main        proc
                ; assign the data segment base address to DS
                mov     ax, data
                mov     ds, ax

                ; TODO ...
               ; | add your code between arrows |
```

```
        ; v ---------------------------- v
        mov     cx, n
        xor     si, si
        xor     di, di
        mov     bx, 0

begin:
        mov     ax, [M + si]
        cmp     ax, 0
        jg      after_abs
        neg     ax

after_abs:
        cmp     ax, bx
        jl      go
        mov     bx, ax
        mov     di, si

go:
        add     si, 2
        loop    begin

        mov     [M + 2 * n], bx
        mov     [M + 2 * (n+1)], di
        ; ^ ---------------------------- ^
        ; |              The END                |

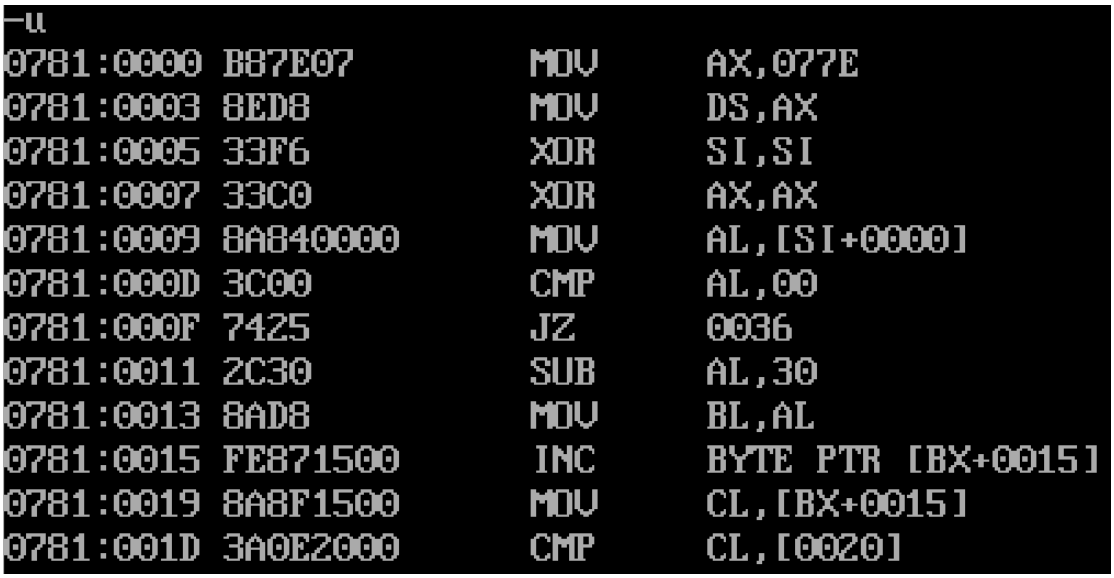
        ; method 2: return to dos
        mov     ax, 4c00h
        int     21h
    main    endp
```

code ends

end main

2

1



```
0781:0021 7210          JB      0033
0781:0023 7706          JA      002B
0781:0025 3A1E1F00      CMP     BL,[001F]
0781:0029 7608          JBE     0033
0781:002B 880E2000      MOV     [0020],CL
0781:002F 881E1F00      MOV     [001F],BL
0781:0033 46            INC     SI
0781:0034 EBD3          JMP     0009
0781:0036 8A161F00      MOV     DL,[001F]
0781:003A 80C230        ADD     DL,30
0781:003D B402          MOV     AH,02
0781:003F CD21          INT     21
```

```
0781:0041 B22C          MOV     DL,2C
0781:0043 CD21          INT     21
0781:0045 8A162000      MOV     DL,[0020]
0781:0049 80C230        ADD     DL,30
0781:004C CD21          INT     21
0781:004E B8004C        MOV     AX,4C00
0781:0051 CD21          INT     21
```

2

```
-d 0 13
077E:0000  32 32 33 36 31 31 34 39-36 38 31 32 33 34 35 36   2236114968123456
077E:0010  37 38 39 30                                       7890
```

3 COUNT COUNT

```
-d 15 1e
077E:0010                       00 00 00-00 00 00 00 00 00 00
```

4

```
-d 0 13
077E:0000  32 32 33 36 31 31 34 39-36 38 31 32 33 34 35 36   2236114968123456
077E:0010  37 38 39 30                                       7890
```

5 COUNT COUNT

```
-d 15 1e
077E:0010                       01 03 03-02 02 01 03 01 02 02
```

6 DOSBox

```
-g
6,3
Program terminated normally
```

7

```
; empty asm file

title I love asm

data segment
    s           db   '2 236 114 968 1 234 567 890', 0
    count       db   10 dup(0)
    max_char    db   0
    max_count   db   0

data ends

code segment
    assume cs:code, ds:data
    main    proc
        ; assign the data segment base address to DS
        mov   ax, data
        mov   ds, ax

        ; TODO ...
        ; | add your code between arrows |
        ; v ---------------------------- v
        xor   si, si
        xor   ax, ax
```

```
scan:
        mov     al, [s + si]
        cmp     al, 0
        je      scan_end

        sub     al, '0'
        mov     bl, al
        inc     count[bx]
        mov     cl, count[bx]
        cmp     cl, max_count
        jb      skip
        ja      max
        cmp     bl, max_char
        jbe     skip
max:
        mov     max_count, cl
        mov     max_char, bl
skip:
        inc     si
        jmp     scan
scan_end:
        mov dl, max_char
        add dl, '0'
        mov ah, 02h
        int 21h

        mov dl, ','
        int 21h

        mov dl, max_count
        add dl, '0'
```

```
        int 21h

        ; ^ ---------------------------- ^
        ; |              The END                |

        ; method 2: return to dos
        mov     ax, 4c00h
        int     21h
    main    endp
code ends
end main
```